

CREACIÓN DEL PROYECTO FRONTEND



Desarrollo del servicio de conexión HTTP en Angular

Introducción

En esta lección aprendimos a crear las clases `Persona` (modelo de datos) y `DataService` (servicio de conexión a Web Services) en Angular. Estas clases nos permitirán comunicarnos con el backend Java previamente implementado, utilizando llamadas HTTP como `GET`, `POST`, `PUT` y `DELETE`.

1. Clase de Modelo: `persona.model.ts`

Comando de Angular:

```
ng g class Persona --type=model --skip-tests
```

<https://www.globalmentoring.com.mx>

Explicación:

- `ng generate class Persona`: crea una clase en Angular.
- `--type=model`: agrega el sufijo `model` al nombre del archivo (`persona.model.ts`).
- `--skip-tests`: evita crear el archivo `.spec.ts` de pruebas.

Ruta del archivo:

`src/app/persona.model.ts`

Descripción:

Creamos una clase modelo llamada `Persona`, que representa los objetos que recibiremos o enviaremos al backend. Incluye dos propiedades: `idPersona` y `nombre`.

Código:

```
export class Persona {  
  
    constructor(public idPersona: number, public nombre: string) { }  
}
```

Explicación:

- `export`: permite que esta clase sea utilizada en otros archivos.
- `constructor(...)`: crea una instancia de `Persona` con dos propiedades.
 - `public idPersona`: identificador numérico de la persona.
 - `public nombre`: nombre de la persona como texto (`string`).
- Al declarar los parámetros como `public`, Angular automáticamente los convierte en propiedades de la clase.

2. Clase de Servicio: `data-service.ts`

Comando actualizado de Angular:

`ng g s data --skip-tests`

Explicación:

- `ng generate service data`: crea el servicio con el nombre `data.service.ts`.
- `--skip-tests`: evita crear el archivo `.spec.ts` de pruebas.

Ruta del archivo:

src/app/data-service.ts

Descripción:

Este servicio contiene todos los métodos necesarios para interactuar con los Web Services RESTful creados en Java. Utiliza `HttpClient` de Angular para hacer las peticiones al servidor.

Código:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Persona } from '../persona.model';

@Injectable({
  providedIn: 'root'
})
export class DataService {

  constructor(private httpClient: HttpClient) {}

  urlBase = 'http://localhost:8080/personas-backend-java/webservice/personas';

  cargarPersonas() {
    return this.httpClient.get(this.urlBase);
  }

  agregarPersona(persona: Persona): Observable<Persona> {
    return this.httpClient.post(this.urlBase, persona);
  }

  modificarPersona(idPersona: number, persona: Persona) {
    const url = `${this.urlBase}/${idPersona}`;
    this.httpClient.put(url, persona)
      .subscribe({
        next: (response) => {
          console.log('resultado modificar persona:', response);
        },
        error: (error) => {
          console.log('Error en modificar persona:', error);
        }
      });
  }
}
```

```

eliminarPersona(idPersona: number) {
  const url = `${this.urlBase}/${idPersona}`;
  this.httpClient.delete(url)
    .subscribe({
      next: (response) => {
        console.log('resultado eliminar persona:', response);
      },
      error: (error) => {
        console.log('Error en eliminar persona:', error);
      }
    });
}
}

```

Explicación:

Decorador @Injectable

- Indica que esta clase puede inyectarse como dependencia.
- { providedIn: 'root' } la hace accesible a toda la aplicación.

Constructor con HttpClient

- `httpClient` es una clase de Angular que permite hacer peticiones HTTP.
- Se inyecta como dependencia para usarlo en los métodos.

`urlBase`

- Contiene la URL base del backend Java que expone los Web Services.
- Se utilizará como base para todas las peticiones.

Método `cargarPersonas ()`

- Realiza una petición `GET` al backend.
- Retorna la lista de personas del servidor.

Método `agregarPersona (persona)`

- Envía una persona al backend con una petición `POST`.
- El parámetro `persona` es el objeto que se va a guardar.

Método `modificarPersona (idPersona, persona)`

- Envía una petición `PUT` para actualizar una persona.
- Concatena el `idPersona` a la URL.
- Se suscribe a la respuesta para mostrar el resultado en consola.

Método `eliminarPersona(idPersona)`

- Envía una petición `DELETE` con el `idPersona` en la URL.
- También se suscribe a la respuesta para mostrar en consola si fue exitoso o hubo error.

Conclusión

Con esta guía logramos implementar correctamente el modelo de datos `Persona` y el servicio `DataService` en Angular. Ahora nuestra aplicación puede comunicarse con los Web Services creados en Java, permitiendo **listar, agregar, modificar y eliminar** personas usando el protocolo HTTP desde el frontend Angular. 🚀

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)